

Документация к GPU Sandbox

Содержание

1	Назначение программы	2
2	Архитектура проекта	2
2.1	Память	2
2.2	Ядро процессора	3
2.2.1	Регистры	3
2.2.2	Программный счётчик	3
2.2.3	Флаги состояния	3
2.2.4	Состояния ядра	3
3	Инструкции виртуального процессора	4
3.1	Параметры инструкций	4
3.2	Поддерживаемые операции	5
4	Симулятор выполнения	6
5	Проверка корректности входных данных	6
6	Отладочный вывод	7
7	Сброс и инициализация	7
8	Тестирование	7

1 Назначение программы

GPU Sandbox представляет собой программную модель вычислительного ядра, памяти и набора инструкций, реализованную на языке C. Цель проекта — изучение основных принципов работы процессора: адресация памяти, выполнение операций в регистрах, обработка состояний ядра, проверка корректности входных данных и симуляция исполнения программы.

На текущем этапе проект уже содержит не только базовые вычисления, но и элементы условных переходов, проверки ошибок, таймаутов, отладки и автоматического тестирования¹.

2 Архитектура проекта

Модель содержит три ключевых компонента: вычислительное ядро, память и программу, представленную набором инструкций.

Таблица 1: Ключевые компоненты архитектуры

Компонент	Назначение
Memory	Виртуальная память, хранящая данные и обеспечивающая чтение/запись по адресам.
Core	Вычислительное ядро с регистрами, счётчиком команд и состояниями выполнения.
Instruction	Структура команды, задающая тип операции и аргументы <i>a</i> , <i>b</i> , <i>c</i> .
Simulator	Цикл выполнения программы с контролем состояния ядра, ошибок и таймаута.

2.1 Память

Memory — это структура виртуальной памяти, хранящая данные в массиве `data[MEMORY_SIZE]`.

Память используется для хранения значений, которые считываются и записываются в ходе выполнения программы. На данный момент размер памяти определяется константой `MEMORY_SIZE = 1024 * 1024`.

Операции чтения и записи реализованы следующим образом:

- `memory_write(Memory *memory, uint32_t address, uint8_t value)` — запись байта в ячейку памяти;
- `memory_read(const Memory *memory, uint32_t address)` — чтение байта из памяти;
- `memory_reset(Memory *memory)` — очистка памяти до нуля.

Проверка допустимости адреса выполняется через специальный валидатор, который запрещает обращение за пределы памяти.

¹Подробнее об устройстве ядра, памяти и логике выполнения см. в [разделе Архитектура проекта](#).

2.2 Ядро процессора

Core — структура вычислительного ядра. В ней хранятся регистры, программный счётчик, текущее состояние и флаг сравнения.

2.2.1 Регистры

`registers[REGISTERS_COUNT]` — массив регистров, который используется для промежуточных вычислений и передачи данных между инструкциями.

Количество регистров задаётся константой `REGISTERS_COUNT`, который, на данный момент, равен 8.

2.2.2 Программный счётчик

`pc` (программный счётчик) — счётчик, который хранит индекс текущей выполняемой команды. После выполнения инструкции счётчик изменяется в соответствии с состоянием ядра.

2.2.3 Флаги состояния

Внутри ядра присутствует переменная `flag`, которая используется для результатов сравнения. В проекте применяются следующие значения:

- `SAME` — значения равны;
- `GREATER` — первое значение больше второго;
- `LESS` — первое значение меньше второго.

2.2.4 Состояния ядра

`current_state` управляет поведением выполнения программы. Набор состояний представлен в перечислении `State`:

1. `NORMAL` — обычный режим работы ядра;
2. `JUMP` — выполнение перехода на другую инструкцию;
3. `HALT` — завершение работы программы;
4. `ERROR` — аварийное состояние при ошибке выполнения;
5. `TIMEOUT` — превышен лимит количества циклов.

При возникновении ошибки, например, вызове некорректного регистра, адреса памяти или неизвестного оператора, состояние переводится в `ERROR`. Если же программа выполняется слишком долго, то срабатывает таймаут по переменной `MAX_CYCLES`.

3 Инструкции виртуального процессора

Структура инструкции представлена типом `Instruction`²:

```
1 typedef struct{
2     Opcode opcode;
3     uint32_t a;
4     uint32_t b;
5     uint32_t c;
6 } Instruction;
```

В проекте используются следующие значения опкодов:

1. OP_LOAD
2. OP_STORE
3. OP_ADD
4. OP_SUB
5. OP_MUL
6. OP_DIV
7. OP_HALT
8. OP_JUMP
9. OP_CMP
10. OP_JIF

3.1 Параметры инструкций

В проекте применяется следующий порядок использования операндов:

- **a** — целевой регистр или регистр с адресом памяти, в зависимости от инструкции;
- **b** — регистр с адресом памяти, исходный регистр или второй аргумент, в зависимости от инструкции;
- **c** — второй источник данных при арифметических операциях.

²Подробное описание наборов операций и их логика работы представлена в [таблице инструкций](#).

3.2 Поддерживаемые операции

Таблица 2: Набор инструкций

Операция	a	b	c	Назначение
OP_LOAD	регистр назначения	регистр с адресом	—	Чтение значения из <code>memory[reg[b]]</code> в <code>reg[a]</code>
OP_STORE	регистр с адресом	регистр со значением	—	Запись значения <code>reg[b]</code> в ячейку <code>memory[reg[a]]</code>
OP_ADD	регистр назначения	регистр 1	регистр 2	Сложение <code>reg[b] + reg[c]</code> и сохранение в <code>reg[a]</code>
OP_SUB	регистр назначения	регистр 1	регистр 2	Вычитание <code>reg[c]</code> из <code>reg[b]</code> и сохранение в <code>reg[a]</code>
OP_MUL	регистр назначения	регистр 1	регистр 2	Умножение <code>reg[b] * reg[c]</code> и сохранение в <code>reg[a]</code>
OP_DIV	регистр назначения	регистр 1	регистр 2	Деление <code>reg[b] / reg[c]</code> с проверкой на деление на ноль
OP_JUMP	индекс инструкции	—	—	Безусловный переход к инструкции с индексом, хранящимся в регистре a
OP_CMP	регистр 1	регистр 2	—	Сравнение значений регистров и установка флага <code>flag</code>
OP_JIF	индекс инструкции	ожидаемый флаг	—	Условный переход: если <code>b == core->flag</code> , ядро переходит по адресу a
OP_HALT	—	—	—	Завершение выполнения программы и перевод ядра в состояние <code>HALT</code>

Важно: фактическая реализация проекта допускает и проверяет корректность входных индексов и адресов, поэтому многие операции в случае неверного операнда переводят систему в состояние `ERROR`.

4 Симулятор выполнения

Модуль `simulator` запускает программу в цикле до тех пор, пока не будет выполнен один из следующих сценариев:³

- достигнут конец программы;
- ядро перешло в состояние `HALT`;
- возникла ошибка;
- число выполненных циклов превысило лимит `MAX_CYCLES`.

В проекте глобальная переменная `cycles` хранит количество уже выполненных инструкций, а константа `MAX_CYCLES`, на данный момент, равна 1 000 000.

Схема выполнения выглядит так:

1. выбирается текущая инструкция по индексу `pc`;
2. она исполняется функцией `execute`;
3. состояние ядра проверяется на наличие ошибки;
4. если состояние `NORMAL`, счётчик увеличивается на единицу;
5. если состояние `JUMP`, оно сбрасывается обратно в `NORMAL`;
6. после каждого цикла увеличивается счётчик `cycles`;
7. при превышении лимита вызывается состояние `TIMEOUT`.

5 Проверка корректности входных данных

Для контроля ошибок проект использует набор валидаторов, реализованных в модуле инструкций⁴. Основные проверки:

- `checker_destination` — проверка допустимого индекса целевого регистра;
- `checker_address_register` — проверка адресного регистра;
- `checker_source_register` — проверка регистра-источника;
- `checker_condition_register` — проверка регистра условия;
- `checker_target_register` — проверка регистра перехода;
- `checker_memory_address` — проверка корректности адреса памяти;
- `checker_instruction_index` — проверка индекса инструкции в пределах программы.

В случае некорректных данных ядро переводится в состояние `ERROR`, а в консоль выводится сообщение о причине сбоя.

³Более подробно логика работы симулятора и условия завершения описаны в [разделе Тестирование](#).

⁴Подробнее о логике ошибок и проверках см. [раздел Симулятор выполнения](#).

6 Отладочный вывод

Проект содержит набор функций для вывода состояния виртуального процессора.⁵

- `print_registers` — печать всех регистров;
- `print_state` — печать текущего состояния ядра;
- `print_program` — отображение программы с операциями и аргументами.

7 Сброс и инициализация

Для повторного использования модели предусмотрены функции сброса:

- `core_reset(Core *core)` сбрасывает регистры в ноль, обнуляет счётчик `pc` и возвращает состояние ядра в `NORMAL`;
- `memory_reset(Memory *memory)` очищает память.

Это позволяет запускать несколько тестовых сценариев последовательно на одном и том же экземпляре модели.

8 Тестирование

В папке `tests` находятся отдельные программы, проверяющие корректность работы ключевых компонентов.⁶

- `test_core.c` — проверяет сброс ядра;
- `test_memory.c` — проверяет запись, чтение и очистку памяти;
- `test_instruction.c` — проверяет все основные инструкции и проверки ошибок;
- `test_debug.c` и `test_simulator.c` — покрывают дополнительные сценарии отладки и симуляции.

Автотесты написаны через макросы `assert`, что позволяет проверять корректность поведения программы на уровне условий и побочных эффектов.

⁵Функции отладки напрямую связаны с режимами работы ядра, описанными в [разделе Архитектура проекта](#).

⁶См. также [раздел Проверка корректности входных данных](#).